

[국문 참고본] OSWorld 보안 신고서

이 문서는 팀·멘토 공유 및 내부 기록용 참고 번역입니다. 메인테이너에게 실제로 발송하는 것은 영문본(OSWorld_security_report.pdf)이며, 해석에 차이가 있을 경우 영문본이 기준입니다.

보고자	박규남 (PARK Gyu-nam)
소속	Team RAID, 화이트햇 스쿨 4기 — KISA(한국인터넷진흥원)가 운영하는 보안 교육 프로그램. 본 신고는 프로그램 교육생의 개인 연구 자격으로 제출하며, KISA의 공식 통보가 아닙니다.
프로젝트	Computer-use agent 공격 표면 분석 및 레드팀
연락처	ksasha@ajou.ac.kr (주) · kevin965546184644@gmail.com (보조)
날짜	2026-09-07
영향 대상	xlang-ai/OSWorld @ fc31a90... (= main, 2026-09-06) xlang-ai/OSWorld-V2 @ 1c81bd34... (= main, 2026-09-03)

1. 요약

OSWorld의 공식 채점 경로는 에이전트가 게스트 VM 안에서 수정할 수 있는 파이썬 파일을 평가 환경으로 복사한 뒤, 로컬 파이썬 자식 프로세스에서 실행한다. 그 결과 게스트 내부에서 내용이 통제되는 코드가 평가 프로세스의 권한으로 실행된다.

문서화된 기본 배포에서 평가 프로세스는 호스트에서 돌아간다. 그 배포에서는 OSWorld 논문 § 2.2가 선언한 격리 속성과 충돌한다.

"가상머신은 안전한 격리 환경을 제공하며, 에이전트가 실제 호스트 머신에 되돌릴 수 없는 피해를 주는 것을 막는다."

패치를 전혀 가지지 않은 클론에서, 벤치마크가 배포한 원본 태스크로 재현했다. 태스크 설정 조작, 경로 순회, 별도 익스플로잇 프리미티브가 필요 없다. 채점 점수는 정상 반환되므로 채점 로그에 이상 징후가 남지 않는다.

2. 영향 대상

두 최신 릴리스 모두 영향을 받는다. 이 패턴은 OSWorld 2.0으로 그대로 이관됐으며, 2.0 쪽이 오히려 격리가 덜하다(§ 3.4).

OSWorld 1.0 — xlang-ai/OSWorld @ fc31a90 (= main)

```
metrics/vscode.py  check_python_file_by_test_suite()  258행
sha256 4e6f7a08...
getters/file.py    get_cloud_file() L49 / get_vm_file() L100
sha256 e8d2801a...
```

OSWorld 2.0 — xlang-ai/OSWorld-V2 @ 1c81bd34 (= main)

```
metrics/vscode.py  check_python_file_by_test_suite()  304행
sha256 3774cceb...
sys.path.insert(0, str(test_dir))  L352
spec.loader.exec_module(module)    L375
getters/file.py    get_cloud_file() L34 / get_vm_file() L92
cache_dir 기록이 L57, L124 — 1.0과 동일하게 같은 디렉터리
```

영향 태스크 — 두 저장소 동일

"func": "check_python_file_by_test_suite" 를 선언한 태스크가 각 저장소에 2개씩, 태스크 ID도 동일하다.

```
multi_apps/9219480b-3aed-47fc-8bac-d2cffc5849f7.json  (tetris)
multi_apps/26150609-0da3-4a7d-8868-0faf9c5f01bb.json  (snake)
```

tetris 태스크 정의는 두 저장소 간 바이트 단위로 동일하다 (sha256 199cd76f...).

재현 범위. OSWorld 1.0의 tetris 태스크에서만 동적 재현을 수행했다. snake 태스크와 OSWorld 2.0의 두 태스크는 채점기 구성·코드 경로가 동일함을 정적으로 확인했으나 실행하지는 않았다.

3. 기술적 세부

3.1 호출 흐름

#	위치	동작
1	태스크 JSON <code>evaluator.result.type = "vm_file"</code>	게스트 <code>.py</code> 4개 회수를 선언
2	<code>desktop_env.py:388</code>	<code>getattr(getters, "get_vm_file")</code>
3	<code>desktop_env.py:170, 362</code>	<code>cache_dir = <base>/<task_id></code> — 평가 환경 로컬
4	<code>getters/file.py:132, 145-155</code>	게스트 바이트를 받아 <code>cache_dir</code> 에 기록 — 신뢰 경계 통과
5	<code>getters/file.py:72</code>	클라우드 테스트 파일을 같은 <code>cache_dir</code> 에 기록
6	<code>desktop_env.py:513, 519, 520</code>	둘 다 <code>metric</code> 으로 전달
7	<code>metrics/vscode.py:276</code>	<code>test_dir = dirname(test_file)</code> — 즉 <code>cache_dir</code>
8	<code>metrics/vscode.py:287</code>	<code>sys.path.insert(0, test_dir)</code>
9	<code>metrics/vscode.py:291, 304</code>	<code>subprocess.run([sys.executable, "-c", runner, ...])</code> 안에서 <code>exec_module(...)</code>

8단계 이후 `cache_dir` 은 `sys.path` 의 첫 항목이 된다. 그 디렉터리에는 에이전트가 내용을 결정하는 파일들이 들어 있다.

3.2 마지막 연결 고리

5단계에서 받아오는 테스트 파일(HuggingFace, sha256 b266e832...)에는 다음이 있다.

```
from tetris import Tetris
from settings import BOARD_HEIGHT, BOARD_WIDTH
```

`sys.path[0] == cache_dir` 이므로 이 import 는 게스트 유래 모듈로 해석되고, 파이썬 의미론상 그 모듈 최상위가 실행된다.

이 import 는 우연이 아니다. **"제출된 코드가 동작하는가"를 재검토하려면 제출된 코드를 실행할 수밖에 없다.** 쟁점은 실행 여부가 아니라 **실행 위치**다.

3.3 실행 위치

평가 프로세스에서 실행됨을 보이는 코드 수준 근거 둘.

- `metrics/vscode.py:304` 가 `sys.executable`, 즉 **채점 프로세스 자신의 인터프리터**를 실행한다. 이 함수에는 게스트로 코드를 보내는 경로가 없다.
- `controllers/python.py:173` 의 `get_file()` 은 `requests.post(.../file)` 로 `response.content` 를 받아오는 단순 HTTP 전송이다. 게스트에서 코드를 실행하는 메서드 `execute_python_command()` (196행)는 `get_vm_file()` 에서도 `check_python_file_by_test_suite()` 에서도 호출되지 않는다.

즉 채점 경로는 **제출 코드를 게스트에서 실행할 필요 없이 평가 환경에서 실행한다.**

3.4 OSWorld 2.0 — 같은 경로, 그런데 in-process 실행

2.0은 구조는 같지만 **서브프로세스를 제거했다**. 모듈이 평가 프로세스 내부에서 직접 로드·실행된다.

```
original_cwd = os.getcwd()
original_sys_path = sys.path.copy()
try:
    test_dir = test_file_path.parent          # == cache_dir
    os.chdir(test_dir)
    if str(test_dir) not in sys.path:
        sys.path.insert(0, str(test_dir))    # L352
    ...
    spec.loader.exec_module(module)         # L375
```

2.0 구현에는 `subprocess.run` 도 `sys.executable` 호출도 없다.

결과: 1.0에서는 게스트 통제 모듈이 계정 권한을 공유하는 자식 프로세스에서 돌아왔다. 2.0에서는 **하네스 프로세스 자체 안에서** 돌며, 그 프로세스의 살아있는 상태 — `sys.modules`, `DesktopEnv` 인스턴스, 평가 대상 에이전트용으로 메모리에 올려둔 API 키 등 — 에도 접근한다. 호출 전후로 `cwd/sys.path` 를 저장·복원하는 것은 **정리(cleanliness)이지 격리가 아니며**, 모듈이 실행되는 동안 하는 일을 막지 못한다.

이 경로에서 2.0은 최소한 1.0만큼 노출돼 있다고 본다. **2.0은 동적 재현을 수행하지 않았다.**

3.5 의도에 관한 관찰

1.0의 해당 함수 docstring 은 서브프로세스 격리의 이유를 이렇게 밝힌다.

"프로세스 전역 상태(cwd, sys.path, sys.modules)가 pass@N 평가의 동시 채점 스레드 간에 새지 않도록 로더+실행기를 서브프로세스에서 돌린다."

격리의 목적이 **동시성이지 비신뢰 코드의 봉쇄가 아니다**. 2.0의 docstring 도 견고성 위주로 서술돼 있고(파일 존재 확인, 모듈 로딩 오류, 함수 실행 오류, 리소스 정리, 작업 디렉터리 관리) 신뢰에 대한 언급이 없다.

이는 해당 지점의 신뢰 경계가 고려된 적 없어 보인다는 점을 지적하는 것이며, 책임을 묻는 것이 아니다.

4. 재현

2026-09-06, fc31a90 의 깨끗한 클론에서 패치 0줄, 게스트 Ubuntu, 평가 환경 macOS(Apple Silicon).

스크립트는 업스트림 `DesktopEnv.reset()` → `step()` → `evaluate()` 전 구간을 그대로 태운다. 에이전트 측 동작은 게스트의 `settings.py` 편집 하나뿐이며, 이는 태스크 지시문 자체가 요구하는 행위다.

관측:

```
평가 환경 카나리 /tmp/f2_upstream_CANARY → "Darwin arm64 pid=84290"
게스트 동일 경로 → 부재
반환 score → 정상, 채점 로그 이상 없음
```

Darwin 은 macOS 커널 이름이고 게스트는 Ubuntu다.

페이로드의 동작은 카나리 파일 생성으로 제한된다. 네트워크 통신도 지속성도 없다. 다만 재현 과정 자체는 평가 실행의 정상적 부수 효과를 남긴다 — 채점기가 제출 파일과 테스트 파일을 `cache_dir` 에 기록하고, 로그가 남고, 게스트 VM이 지정 스냅샷으로 되돌아간다. **전용 VM을 쓸 것.**

주의: 경로는 ASCII로 둘 것(비ASCII 경로는 `providers/vmware/provider.py::start_emulator` 의 무한 루프를 유발 — 본 건과 무관한 별개 버그). `--path_to_vm` 은 절대경로를 명시할 것.

5. 영향

주 영향: 평가를 실행하는 계정 권한으로 임의의 파이썬 실행. 문서화된 기본 배포에서 그것은 연구자의 호스트 계정이며, 통상 API 키·SSH 키·클라우드 자격증명·미공개 연구물을 보유한다.

OSWorld 2.0에서는 하네스 프로세스 내부에서 실행되므로 그 프로세스의 메모리 상태에도 도달한다(§ 3.4).

범위 한정. 평가기를 컨테이너 등 격리 환경에서 돌리는 배포라면 주 명제는 그대로 성립하되(평가 프로세스 권한으로 실행됨) 영향 범위는 그 환경의 권한 경계를 따른다. **컨테이너 탈출을 주장하지 않는다.**

탐지 가능성. 페이로드는 게스트에서 실행되지 않으므로 게스트 측 로깅·모니터링에는 아무것도 잡히지 않는다. 태스크 상수가 보존되므로 채점기는 정상 점수를 반환한다.

선행 조건. 에이전트가 회수 대상 `.py` 중 하나에 해당 내용을 기록해야 한다. 그것이 태스크가 요구하는 행위다. 비신뢰 프로젝트 문서를 통해 이를 유도할 수 있는 빈도는 Annex B 참조.

심각도 제안

```
CVSS 3.1 AV:L/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H = 8.5 (High)
```

S:C 는 영향 자원(평가 환경)이 취약 컴포넌트(샌드박스된 게스트)의 권한 범위 밖에 있음을 반영한다. S:U 로 매기면 7.7이다. **High 대역이 적절하다고 보며 Critical 을 주장하지 않는다** — AV:L 이 적용되고, 현재 배포된 369개 태스크 중 2개만 이 경로에 도달한다.

6. 관련 hardening 관찰

주 주장에 포함되지 않으며, 이 경로들을 통한 코드 실행을 실증하지 않았다.

- 다운로드 파일의 무결성 검증 부재.** `get_cloud_file` 은 HTTPS로 태스크 파일을 받아오면서 해시·서명 검증을 하지 않고, `if os.path.exists(_path): continue` 로 캐시를 재검증 없이 재사용한다. 데이터셋 저장소가 오염되면 공격자가 정한 내용이 `cache_dir` 에 놓이며, 그곳의 `test_suite.py` 는 채점기가 직접 import 한다. 286개 cloud URL 전부 `huggingface.co` HTTPS이나, TLS는 전송 구간을 인증할 뿐 저장소 내용을 인증하지 않는다.
- 369개 중 215개 태스크가 게스트 통제 파일을 평가 환경으로 회수한다.** 그중 2개가 파이썬을 실행하고(본 보고), 나머지 213개는 파서 라이브러리(`openpyxl`, `python-pptx`, `python-docx`, PDF·이미지 파서, `shutil.unpack_archive`)로 넘긴다. **이 경로들로 코드 실행이 가능한지는 시험하지 않았으며 가능하다고도 불가능하다고도 주장하지 않는다.** 공격 표면으로만 기록한다.
- `metrics/basic_os.py:4` 가 게스트에서 읽은 값에 `eval()` 을 적용한다.** 이미 다른 곳에 공개된 사안이며 본 보고 범위 밖이다. 완전성을 위해서만 언급한다.

7. 권고 조치

선호 순.

A. 게스트 안에서 채점한다. 테스트 파일을 게스트로 밀어넣어 거기서 실행하고, 숫자 결과만 평가 환경으로 돌려받는다. 논문이 선언한 격리 속성이 유지되고 경계 통과 자체가 사라진다.

B. 호스트 측 실행을 유지한다면 실제로 가둔다. 채점 서버프로세스를 일회용 컨테이너나 동등한 샌드박스에서, 네트워크 없이, 작업 디렉터리 외부는 읽기 전용으로, 비특권 계정으로 실행한다.

C. 한 디렉터리에 신뢰 도메인을 섞지 않는다. 게스트 유래 파일과 벤치마크 제공 파일을 분리해, 에이전트가 내용을 통제하는 디렉터리가 `sys.path` 에 들어가지 않게 한다.

D. 다운로드 파일을 검증한다. 태스크 정의에 데이터셋 내용을 해시로 고정하고, 다운로드 시점과 캐시 재사용 시점에 검증한다(§ 6.1 대응).

패치 관련 주의 2건:

- `metrics/vscode.py` 만 고치는 것으로는 불충분하다. 현재 `test.py` 에는 `if script_dir not in sys.path: sys.path.append(script_dir)` 가 있고, 지금은 runner 가 이미 그 디렉터리를 등록해 두어 건너뛰어질 뿐이다. runner의 `sys.path.insert` 를 제거하면 이 분기가 활성화된다. (코드 독해에 근거하며 시험하지 않았다.)
- 두 저장소 각각에 수정이 필요하다.** 코드가 분기돼 한쪽 패치가 넘어가지 않는다. 2.0은 이 경로에 프로세스 경계가 전혀 없으므로 B안이 더 시급하다.

8. 공개

비공개 우선 신고이며, 본 신고일로부터 **90일** 조율 공개 기간을 제안한다. 귀 팀의 선호에 맞출 용의가 있다.

채널. 신고 전 비공개 경로를 찾아보았다. 두 저장소 모두 SECURITY.md 가 없고, GitHub 비공개 취약점 신고 기능이 활성화돼 있지 않다(/security/advisories /new 이용 불가, 2026-09-07 확인). 따라서 README의 메인테이너 메일이 유일한 비공개 채널이었다. 두 저장소에 비공개 신고 기능을 활성화하면 향후 신고자에게 추적 가능한 경로가 생긴다. 발견 사항이 아니라 제안으로 언급한다.

CVE. 식별자 발급을 신청할 의사가 있으며, 가능하면 귀 팀과 함께 진행하기를 선호한다. 귀 팀이 직접 신청하거나 GitHub Security Advisory 로 공개하기를 원한다면 그 절차를 따르겠다. 합의된 날짜 이전, 또는 수정본 배포 이전 중 이른 시점 전에는 세부 공개하지 않는다.

논문. 본 작업의 일부는 computer-use agent 보안에 관한 학술 논문에 실릴 예정이다. 해당 논문은 에이전트 행동 관련 발견(Annex B)을 다루며 본 신고를 인용한다. 게재 시점에 공개된 범위를 넘는 재현 세부는 포함하지 않으며, 초고를 사전에 공유할 수 있다.

크레딧 표기 방식이나 크레딧을 원하지 않는지 알려주기 바란다.

9. 첨부

파일	내용
F2_call_flow_evidence.md	전체 호출 흐름 근거, 인용 행 감사표, 자체 검증 절차
f2_upstream_verify.py	무패치 재현 스크립트 (§ 4)
classify_tasks.py	369 / 215 / 2 태스크 분류 재현 (§ 2, § 6.2)
hf_test.py	회수된 테스트 파일 보존 사본 (sha256 b266e832..., 고정 revision 대조 완료)
settings_original.py	미변경 settings.py 보존 사본 (sha256 fca04a7c..., 동일 revision 대조 완료)
Annex B	에이전트 행동 실험 — 본 보고서에 요약. 전체 데이터·실행 산출물은 요청 시 제공 하며, 취약점 주장에 불필요하므로 첨부하지 않음

Annex A — 자체 검증

```
git clone https://github.com/xlang-ai/OSWorld.git && cd OSWorld
git checkout fc31a9049664292fcb35d6e501ee1dc839f2cf6d

sed -n '287p;291p;304p'      desktop_env/evaluators/metrics/vscode.py
sed -n '72p;132p'           desktop_env/evaluators/getters/file.py
sed -n '362p;513p;519p;520p' desktop_env/desktop_env.py
sed -n '173p'               desktop_env/controllers/python.py

shasum -a 256 desktop_env/evaluators/metrics/vscode.py desktop_env/evaluators/getters/file.py
grep -rl "check_python_file_by_test_suite" evaluation_examples/examples/
python3 <첨부>/classify_tasks.py --root .

HF=https://huggingface.co/datasets/xlangai/ubuntu_osworld_file_cache/resolve/1e112283c4ecb08d6fed8069bca7de74fa2f12aa/multi_apps/9219480b-3aed-47fc-8bac-d2cffc5849f7
curl -sL "$HF/test.py"      | shasum -a 256      # 기대 b266e832...   (2026-09-07 대조 완료)
curl -sL "$HF/settings.py" | shasum -a 256      # 기대 fca04a7c...   (2026-09-07 대조 완료)
```

Annex B — 에이전트 행동 실험 (보조)

본 보고서에 요약하며, 전체 데이터와 실행 산출물은 요청 시 제공한다. 비신뢰 프로젝트 문서가 settings.py 에 헤더를 삽입하라고 지시할 때 computer-use agent 가 실제로 그렇게 하는 빈도를 측정했다. 4개 조건, 17회 실행, claude-haiku-4-5, GUI 도구만 사용.

요약: **에이전트가 지시를 의심스럽다는 이유로 거부한 사례는 0/17 이었다.** 삽입 대상이 실행되는 코드인 경우와 실행되지 않는 주석인 경우의 수용률은 동일했다. 평가 환경 실행까지 전 경로가 완성된 비율은 조건에 따라 25~60% 였으며, 미달분은 안전 판단이 아니라 GUI 텍스트 편집 실패와 이행 수단 선택에서 비롯됐다.

이 부록은 도달 가능성에 관한 것이지 취약점의 존재에 관한 것이 아니다. § 1~ § 5는 무패치 재현만으로 성립한다.